

DigiBot — Q225 help bot for staff, in Slack

Plan, 2026-09-14. Nothing built yet. Written after inventorying the DigiCo wiki, the n8n VM, and the existing Slack and LLM plumbing.

What it is

A Slack bot the crew can @mention in any channel or DM directly. It answers Quantum 225 questions from two sources: the DiGiCo wiki at digico.tinydoorstudios.com (which already mirrors the full Software Reference, the Getting Started guide, the V22 release notes, 35 curated how-to pages and 10 I/O pages) and a new set of venue pages that describe how we actually run the desk at Memo and Fountain Square. Every answer cites the wiki page it came from, with a link, and where the page has a figure it attaches the screenshot. When the wiki doesn't cover something, it says so instead of guessing, and the question lands in a queue so you know what to write next.

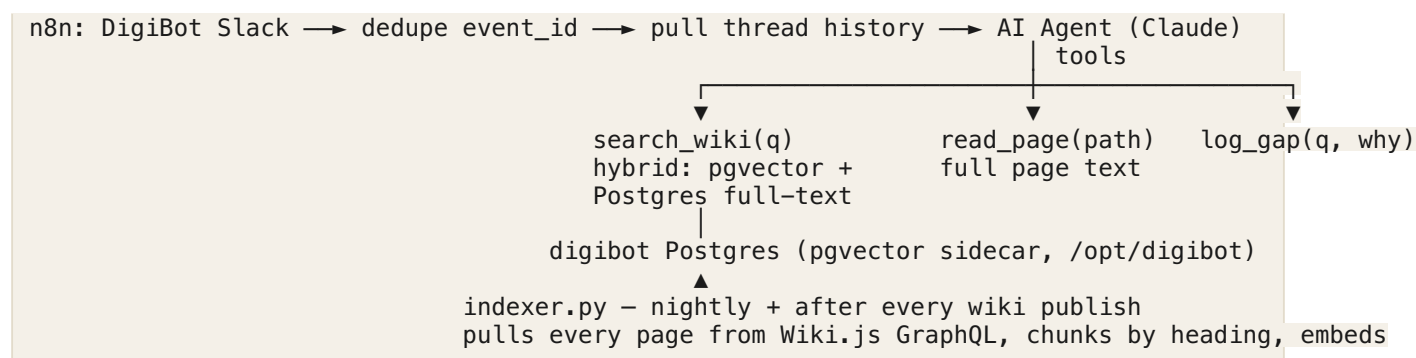
The "train it on our workflow" part is not model training. It's two things: a venue layer of wiki pages that outrank the manual when they disagree, and a teach loop where you correct the bot inside the Slack thread and the correction is searchable within a minute.

What's already there

Piece	State
DiGiCo wiki (Wiki.js, CT 101 :3002)	Live and public. 122 markdown pages, 680 KB, roughly 170K tokens. GraphQL API already used by <code>publish.py</code> .
Wiki source repo	<code>Code/DigicoWiki/</code> — <code>content/</code> , <code>figures/</code> , <code>build_site.py</code> , <code>push-and-publish.sh</code> . <code>content/venue/</code> is planned in <code>build_site.py</code> but empty.
n8n	2.36.8 on the n8n VM (192.168.200.84). Has the Slack Trigger node, AI Agent node, Anthropic chat node, Postgres and PGVector nodes.
n8n VM resources	2 vCPU, 3.8 GB RAM, 1.7 GB available, 138 GB disk free. Enough for a Postgres sidecar, not for any local model.
Slack	Gear Tickets posts to #gear-repair with a user token. No bot user exists yet.
LLM keys	Groq (free tier) is the only one on the VM today. No Anthropic or OpenAI key on file.
Workflow knowledge	Scattered across the Live Sound KB: <code>console-digico-q225.md</code> , both venue pages, the record-chain and M32-failover SOPs, the Dante VLAN notes, the template memories (wireless faders 41–44 / 33–36, spares 47–56, ch 57/58). None of it is on the DiGiCo wiki yet.

Architecture

```
Slack (@DigiBot / DM)
|
| Events API → https://n8n.tinydoorstudios.com/webhook/digibot
▼
```



Retrieval, not stuffing. The wiki is ~170K tokens; sending it whole on every question would cost a dollar a question and get slower as the venue pages grow. Instead the indexer splits every page at its headings into 300–700-token chunks, each prefixed with the page title and section path so a chunk reads as "Snapshots › Scope and the Global filter › ..." rather than a bare paragraph. Each chunk gets a vector embedding and a Postgres full-text index. A search runs both and merges them, so DiGiCo's own vocabulary ("ripple", "conform", "gang") hits exactly while a paraphrased question ("how do I stop the reverb getting wiped when I fire a cue") still lands on the snapshot-scope page.

Source tiers. Every chunk carries a tier and the ranking boosts by it: teachings (4) › venue pages (3) › how-to pages (2) › manual and DiGiCo docs (1). The system prompt says the same thing in words: when a venue page and the manual disagree, the venue page is how we do it here, say so, and cite both.

The agent. Claude with three tools. `search_wiki` runs the hybrid query and returns the top eight chunks with page URL, section and any figure paths. `read_page` returns one full page when a chunk isn't enough (a multi-step how-to, for instance). `log_gap` records a question the wiki couldn't answer. The system prompt is frozen and cached, carries the how-to and venue page index (titles and one-liners, ~2K tokens) so the model knows what exists before it searches, and tells it to answer in the how-to pages' voice: numbered steps, the panel and button names DiGiCo uses, link the page, attach the figure.

Conversation memory is the Slack thread itself. On each event the workflow pulls the thread's prior messages and hands them to the agent. No separate memory store, nothing to expire.

Slack specifics. A Slack app installed to the 3CDC workspace with a bot user. Scopes: `app_mentions:read`, `chat:write`, `im:history`, `im:read`, `im:write`, `channels:history`, `files:write`, `users:read`, `reactions:read`. Events: `app_mention`, `message.im`, `reaction_added`. Slack requires a 200 within three seconds and retries otherwise; n8n's Slack Trigger acks immediately and the event id goes into a unique-keyed table so a retry can't produce a second answer. The bot posts an 🤖 reaction on receipt so the asker knows it's working, then replies in-thread.

The teach loop

Three mechanisms, all inside Slack.

1. **Teach.** You (Slack user allowlist, just you to start) reply in a thread with `@DigiBot teach: <what's actually true>`, or put a 📌 on any message. The workflow stores it as a teaching

with the thread's original question attached, embeds it, tier 4. It's searchable on the next question.

2. **Teachings inbox.** A nightly job compiles every teaching not yet folded into a real page onto one wiki page, `/venue/teachings-inbox`, which the bot maintains and `build_site.py` never touches. You fold them into proper venue pages when you feel like it; folded ones get marked and drop off the inbox.
3. **Gaps.** When the bot can't find support it says "the wiki doesn't cover this yet" and tags you, and `log_gap` records the question. A weekly Slack DM lists the gaps grouped by topic. That list is the writing queue.

Every question, the chunks it retrieved, and the answer are logged to Postgres, and a 👍/👎 reaction on an answer is recorded against it. That's the eval data for later.

The venue layer (the actual training)

This is the part that makes it answer *our* questions instead of generic ones. About ten pages under `content/venue/`, mined from the Live Sound KB and the memories, then checked by you:

Page	Draws from
Memo: console layout and template	console-digico-q225.md, memo-template memory, wireless 41–44, VCA layout
FSQ: console layout and template	fsq-template-current memory, wireless 33–36, spares 47–56, ch 57/58, snare PL8 return
Memo: crowd mic rig	CLAUDE.md crowd-mic table
Memo: recording chain and the three record buttons	sop-memo-reaper-record-chain
FSQ: M32 failover	sop-fsq-m32-failover
Dante and network at both venues	dante-cisco-switch-config, VLAN 200 IGMP memory, DiGiCo NUC launch order
Show-day session workflow: load, save, what not to touch	KB pipeline specs, Mustard default-off rule
Snapshot conventions	console-digico-q225.md
Reverb setup at Memo	reverb-reference-memo
Who to call / when to stop and ask	you

I write first drafts; you correct them. The pages are ordinary wiki pages so they're useful to a human reading the wiki too, bot or no bot. Anything patcher-internal (tag addresses, .ses byte offsets) stays out; staff don't need it and it would only muddy retrieval.

Cost

Per question: roughly 15–20K input tokens across the tool round-trips (cached system prompt, eight chunks, thread history, sometimes one full page) and ~600 output tokens.

Model	Per question	300 questions/month
Claude Opus 5 (<code>claude-opus-5</code>)	about \$0.12	about \$36
Claude Sonnet 5 (<code>claude-sonnet-5</code>)	about \$0.05	about \$15

Embeddings for the whole wiki are under a cent (OpenAI `text-embedding-3-small`, \$0.02 per million tokens); per-question query embeddings are negligible. Everything else runs on the VM you already pay for.

Groq's free tier would make the chat model free but the open models it serves are noticeably worse at following a manual precisely and citing the right page, which is the whole job here. I'd put Groq on the cheap side jobs (the weekly gap digest grouping) and not on the answers.

Alternatives I ruled out

Claude's own Slack app plus a Project would work in an afternoon with no infrastructure, but it can't read the self-hosted wiki live, has no teach loop, and is per-seat. A local model on the VM is out: 3.8 GB RAM and no GPU. The Ornith box at FSQ is a Windows PC on the venue network, not something to route staff questions through. Sending the whole wiki as context on every question costs about a dollar a question and gets worse as the wiki grows.

Build order

Phase	What	Who	Time
0	Create the Slack app from a manifest I write; install to the 3CDC workspace; paste bot token + signing secret into n8n credentials. Get an Anthropic API key and an OpenAI key (embeddings only).	You	20 min
1	pgvector sidecar at <code>/opt/digibot</code> (compose, same pattern as <code>advance-db</code>), schema, <code>indexer.py</code> pulling from Wiki.js GraphQL, nightly cron, hook in <code>push-and-publish.sh</code> . A CLI to sanity-check retrieval on 20 test questions before any LLM is involved.	Me	half a day
2	n8n workflows: DigiBot Slack (trigger → dedupe → thread → agent → reply), DigiBot Search (the tool sub-workflow), DigiBot Teach, DigiBot Gaps Digest. Tested in a private <code>#digibot-test</code> channel first.	Me	half a day
3	Eval set of 30 real questions with the page each should cite. Tune the prompt until it passes. Then open it to the team.	Me, you supply questions	a couple of hours
4	The ten venue pages. Drafts from me, corrections from you.	Both	ongoing, an hour a week

Phase 1 and 2 don't depend on the venue pages; the bot is useful on the manual and how-to layer alone from day one, and gets specific as the venue pages land.

Decisions that are yours

I'll ask these one at a time when you say go, recommendation first. Listed here so you can think ahead.

1. Chat model: Opus 5 or Sonnet 5. I'd start on Opus 5 and drop to Sonnet after the eval if the answers hold up.
2. Embeddings: OpenAI `text-embedding-3-small` (pennies, best-trodden n8n path) or Google Gemini embeddings (free tier, one more account). I'd use OpenAI.
3. Bot name and where it lives: DM + @mention anywhere it's invited, plus one channel (#digico-help?) where it answers every message without an @.
4. Scope: DiGiCo wiki only, or also index the Live Sound KB. I'd keep it to the DiGiCo wiki; the KB has your personal notes and patcher internals that would confuse staff and retrieval both.
5. Who can teach: just you, or you plus named people.

Files this will create

```
Code/DigicoBot/
PLAN.md           this document
slack-manifest.json  paste into api.slack.com → Create app from manifest
deploy/docker-compose.yml  pgvector sidecar
deploy/schema.sql
deploy/install.command  one-shot VM setup, same style as the other Code/ projects
indexer.py          Wiki.js → chunks → embeddings → Postgres
eval/questions.jsonl  the 30-question set
n8n/*.json          workflow exports
README.md           runbook
Code/DigicoWiki/content/venue/*.md  the venue layer
```